

# SATARK-MPLADS: Technical Viva & Cross-Examination Master Blueprint

A Grounded Architectural, Engineering, API, and Fraud-Detector Defense Manual for National Hackathon Evaluations

## Ground Truth & Codebase Reconciliation Notice

Before defending the system, note this key architectural nuance verified from the [ [suvendu-2006/mplads-intelligence-portal](#) ](file:///Users/suvendu/Downloads/SIH-DATA) codebase:

- **The e-SAKSHI vs. PFMS Ingestion Reality:**

The proposal describes cross-checking "e-SAKSHI (work diary records) against PFMS (bank/treasury records)". In the current production codebase:

- SATARK ingests **official MoSPI published open datasets** ( `data/works_completed.csv` , `data/works_completed_detailed.csv` , `data/expenditures.csv` , `data/works_recommended.csv` ).
- In the ORM schema ( `mplads_fraud_detection/foundation/schema.py` ), the `payment_mode` column defaults to `"PFMS/Treasury"` .
- The system performs forensic triangulation by reconciling the **work execution diary records** ( `works` table: 56,896 records) against the **liquid expenditure vouchers** ( `expenditures.csv` / `payment_vouchers` : 29,001 records).
- There are **22,294 flagged anomalies** persistently indexed in `mplads_dev.db` .

## Section 1: API Fundamentals (Core Definitions & Application)

### Q1.1: What is an API endpoint generically? Define "start point" and "end point".

- **API Endpoint:** A distinct, network-addressable Uniform Resource Identifier (URI) paired with an HTTP method ( `GET` , `POST` , `PUT` , `DELETE` ) through which clients communicate with server resources.
- **Start Point (Request):** The entry point initiated by the client. It contains:
  1. The target resource path (e.g., `/api/flags` ).
  2. The HTTP verb ( `GET` ).
  3. Query parameters (e.g., `?page=1&tier=red&detector=cost_overrun` ).
  4. HTTP request headers ( `Authorization` , `Accept-Encoding: gzip` ).
  5. Optional request body/payload.
- **End Point (Response):** The terminal stage where the server completes execution and emits structured output. It contains:
  1. An HTTP status code ( `200 OK` , `400 Bad Request` , `404 Not Found` ).
  2. Response headers ( `Content-Type: application/json` , `Cache-Control` ).
  3. The serialized response payload (JSON data object or stream).

### Q1.2: How does this apply specifically to SATARK-MPLADS?

- **Start Point in SATARK:** When an auditor navigates to the Vigilance Desk to inspect critical cost overruns, the browser dispatches:

```
GET /api/flags?page=1&page_size=50&tier=red&detector=cost_overrun HTTP/1.1
Host: mplads-intelligence-portal.vercel.app
Accept: application/json
Accept-Encoding: gzip
```

- **End Point in SATARK:** FastAPI catches this via `webapi/routers/flags.py`, executes an indexed query on `mplads_dev.db`, compresses the payload via `GZipMiddleware`, attaches Vercel Edge caching headers, and terminates with:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: public, max-age=300, stale-while-revalidate=86400

{
  "data": [ ...50 flagged anomaly objects with CPWD calculations... ],
  "meta": { "page": 1, "page_size": 50, "total": 22294, "total_pages": 446 }
}
```

## Section 2: Frontend — End-to-End Lifecycle

### Q2.1: Trace the full request lifecycle on the client side from user click to screen paint.

```
User Clicks Filter/Tab → React Router Route Match → Lazy-Loaded Page Chunk Mounted
|
▼
Component Renders Loading Skeleton → useEffect Hook Dispatches Fetch via api.ts
|
▼
HTTP Request Emitted to /api/* → JSON Response Received & Parsed
|
▼
Session Storage Updated → React useState/Zustand Updated → DOM Painted
```

1. **User Action:** The user clicks a state card (e.g., *Odisha*) or filters anomalies on the Audit Desk.
2. **Routing:** `react-router-dom` in `web/src/App.tsx` intercepts the path and renders the lazy chunk:  

```
const StateDetail = React.lazy(() => import('./pages/StateDetail'))
```
3. **Mount & Skeleton:** The component displays components while network requests are in flight.
4. **Data Fetching:** A `useEffect` hook invokes the centralized HTTP wrapper in `web/src/lib/api.ts`.
5. **Session Cache Check:** For repeat visits, the app reads from `sessionStorage` (e.g., `cached_audit_flags_1`) to provide zero-latency immediate rendering while revalidating in the background.
6. **State Mutation:** When the JSON arrives, component hooks (`setFlags()`, `setMeta()`) update local state; Zustand store (`useStore.ts`) updates global filters and active user roles.
7. **DOM Render:** React reconciles the virtual DOM, updating metric chips, elements, and SVG chart lines.

### Q2.2: Which libraries render which parts of the interface, and where does each page's data originate?

- **Framework:** React 18 with Vite 5 and TypeScript.
- **Styling:** Tailwind CSS with Lucide React iconography.
- **GIS Map ( /map ):** `leaflet` and `react-leaflet` rendering GeoJSON layers from `/api/map/pcs` (543 Parliamentary Constituencies) and `/api/map/districts` (594 Districts).
- **Charts:** Bespoke SVG data visualization primitives (`web/src/components/charts/`):
  - `TrendArea.tsx` for annual expenditure trajectories.
  - `PremiumDonut.tsx` for civil sectoral distributions.

- `GroupedBars.tsx` for state-wise allocation vs. disbursement bars.
- `CPWDGauge.tsx` for tolerance threshold overrun dials.
- **State Management:** `zustand` ( `web/src/store/useStore.ts` ) tracking authentication and active user roles.

---

## Section 3: Backend — End-to-End Lifecycle

---

**Q3.1: Trace the lifecycle of an incoming request inside the backend service.**

```
Incoming TCP Connection → Uvicorn ASGI Server
|
▼
GZipMiddleware (Compresses responses > 1000 bytes)
|
▼
Performance Cache Middleware (Attaches Cache-Control: max-age=300, stale-while-reval)
|
▼
CORSMiddleware (Validates origin: localhost, *.vercel.app)
|
▼
APIRouter (Resolves route: /api/national, /api/flags, /api/states, etc.)
|
▼
Dependency Injection: SessionLocal (SQLAlchemy session for mplads_dev.db)
|
▼
SQL Query Execution (Indexed filters: LIMIT, OFFSET, WHERE state = ...)
|
▼
Pydantic Schema Serialization (webapi/models.py)
|
▼
Response Serialization → HTTP 200 JSON
```

**Q3.2: Where do validation, authentication/RBAC, and error handling take place?**

- **Validation:** Enforced declaratively via **Pydantic v2 schemas** in `webapi/models.py` and FastAPI query parameter constraints (e.g., `page: int = Query(1, ge=1)`). Malformed queries return structured `422 Unprocessable Entity` responses automatically.
- **Authentication & RBAC:** Managed in `webapi/routers/roles.py` and `webapi/auth_demo.py`. SATARK enforces 4 discrete security roles:
  1. `public_citizen`: Unrestricted transparency view; write actions locked.
  2. `district_authority`: Scoped to specific district; grants access to `/api/districts` approval actions.
  3. `state_nodal_officer`: Scoped to specific state; views multi-district aggregate pipelines.
  4. `auditor`: CAG / MoSPI vigilance inspector; full access to the 22,294 anomaly dossiers.
- **Error Handling:** Handled via Python `try...except` blocks wrapped in each router, emitting explicit `fastapi.HTTPException(status_code=404, detail="...")` with fallback snapshots to ensure the API never crashes with unhandled 500 errors.

---

## Section 4: Data Flow Map Across Major Screens

---

| Screen / Feature                   | Data Shown                                                                                        | API Endpoint                                             | Underlying Storage / Table                                   | Transformation in Between                                                                                |
|------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| National Dashboard ( / )           | Total allocated (₹11,682 Cr), disbursed (₹3,964 Cr), queue (40,233), gap (39.8%), sectoral spend. | GET /api/national/summary<br>GET /api/national/analytics | all_districts_mplads_summary.csv & works table               | In-memory aggregation across all 36 States/UTs; sectoral grouping into roads, lighting, community halls. |
| State Directory ( /states )        | 36 States/UTs, allocation vs. disbursal, utilization %.                                           | GET /api/states                                          | all_districts_mplads_summary.csv                             | Group-by state; sorting by allocated capital; calculating completion-to-queue ratios.                    |
| State Detail ( /states/:id )       | District allocations, MP representation, state works ledger.                                      | GET /api/states/{state}                                  | works table & all_districts_mplads_summary.csv               | SQL WHERE state = :state , grouping by district, calculating local absorption.                           |
| MP Browser ( /mps )                | 774 MPs, House, term, allocated funds, utilization rate.                                          | GET /api/mps                                             | all_mps_summary.csv & all_mps_financial_breakdown.csv        | Search filtering ( q ), house filtering ( LS / RS ), sorting by utilization.                             |
| District Console ( /district/:id ) | District sanction portfolio, implementing agencies (IDAs), project ledger.                        | GET /api/districts/{district}                            | works table ( mplads_dev.db )                                | Filtering WHERE district = :district , calculating execution velocity ratio.                             |
| Audit Desk ( /audit )              | 22,294 flagged works, CPWD rates, March surges, split bills.                                      | GET /api/flags<br>GET /api/flags/{id}                    | anomalies table ( mplads_dev.db )                            | Paginated SQL queries ( LIMIT/OFFSET ), string matching, JSON evidence unpacking.                        |
| GIS Map ( /map )                   | 543 PCs & 594 Districts choropleth boundaries and telemetry.                                      | GET /api/map/pcs<br>GET /api/map/districts               | data/pcs_enriched.geojson<br>data/districts_enriched.geojson | GeoJSON streaming with GZip compression and client-side layer caching.                                   |

## Section 5: Exhaustive API Endpoint Directory

| Method | Path (Start Point)        | Parameters / Body                              | Returns (End Point)                                            | Frontend Caller          |
|--------|---------------------------|------------------------------------------------|----------------------------------------------------------------|--------------------------|
| GET    | /api/national/summary     | None                                           | National KPI dictionary (outlay, spent, works, rate).          | NationalDashboard.tsx    |
| GET    | /api/national/analytics   | None                                           | Sectoral breakdown array & multi-year trend curve.             | NationalDashboard.tsx    |
| GET    | /api/states               | sort , order                                   | Array of 36 State/UT financial objects.                        | BrowseStates.tsx         |
| GET    | /api/states/{state}       | Path: state (string)                           | State summary, district list, and works summary.               | StateDetail.tsx          |
| GET    | /api/districts/{district} | Path: district (string)                        | District sanction portfolio, IDA distribution, works.          | DistrictDashboard.tsx    |
| GET    | /api/mps                  | q , house , sort , order                       | Array of MP financial and biographical records.                | BrowseMPs.tsx            |
| GET    | /api/mps/{mp_id}          | Path: mp_id (string)                           | Detailed MP financial ledger and recommended works.            | MPDetail.tsx             |
| GET    | /api/flags                | page , page_size , q , state , tier , detector | Paginated anomaly objects + pagination metadata.               | AuditDesk.tsx            |
| GET    | /api/flags/{work_id}      | Path: work_id (int)                            | Single flag evidence dossier + CPWD benchmark data.            | FlagDossierModal.tsx     |
| GET    | /api/entity-risks         | entity_type , state , page                     | Risk rankings for IDAs or MPs.                                 | AuditDesk.tsx            |
| GET    | /api/map/pcs              | None                                           | GeoJSON FeatureCollection of 543 Parliamentary Constituencies. | GISMap.tsx               |
| GET    | /api/map/districts        | None                                           | GeoJSON FeatureCollection of 594 Districts.                    | GISMap.tsx               |
| GET    | /api/roles/me             | None                                           | Active user role profile and permissions.                      | Navbar.tsx , useStore.ts |
| POST   | /api/roles/switch         | Body: {"role": "..."}                          | Updated session role confirmation.                             | SwitchRoleDropdown.tsx   |
| GET    | /api/meta/detectors       | None                                           | Array of D1–D15 detector metadata and status.                  | AuditDesk.tsx            |

## Section 6: Key Performance Indicators (KPIs)

### Q6.1: What is a KPI generically?

- **Generic Definition:** A quantifiable metric used to evaluate performance against a strategic, operational, or statutory standard.

### Q6.2: What actual KPIs does SATARK track, and how is each calculated?

#### 1. Total Allocated Budget (\$\text{₹11,682 Cr}\$):

$$\text{Total Allocated} = \sum \text{Central Government Installment Sanctions to all States/UTs}$$

#### 2. Used (Disbursed) Outlay (\$\text{₹3,964 Cr}\$):

$$\text{Used Outlay} = \sum \text{Liquid Disbursements Released to Implementing Agencies}$$

#### 3. National Utilization Rate (\$33.9\%\$):

$$\text{Utilization Rate} = \left( \frac{\text{Used Outlay}}{\text{Total Allocated}} \right) \times 100$$

#### 4. Payment Gap (\$39.8\%\$):

$$\text{Payment Gap} = \left( \frac{\text{Allocated Outlay} - \text{Disbursed Outlay}}{\text{Allocated Outlay}} \right) \times 100$$

*Measures funds legally sanctioned by Parliament but stalled in treasury pipelines.*

#### 5. Works Completion Ratio:

$$\text{Completion Ratio} = \frac{\text{Works Completed (43,735)}}{\text{Total Recommended Works (83,968)}} = 52.1\%$$

#### 6. CPWD Rate Escalation Delta (\$\Delta C\$):

$$\Delta C = \frac{\text{Actual Billed Unit Price} - \text{CPWD Ceiling Rate}}{\text{CPWD Ceiling Rate}} \times 100$$

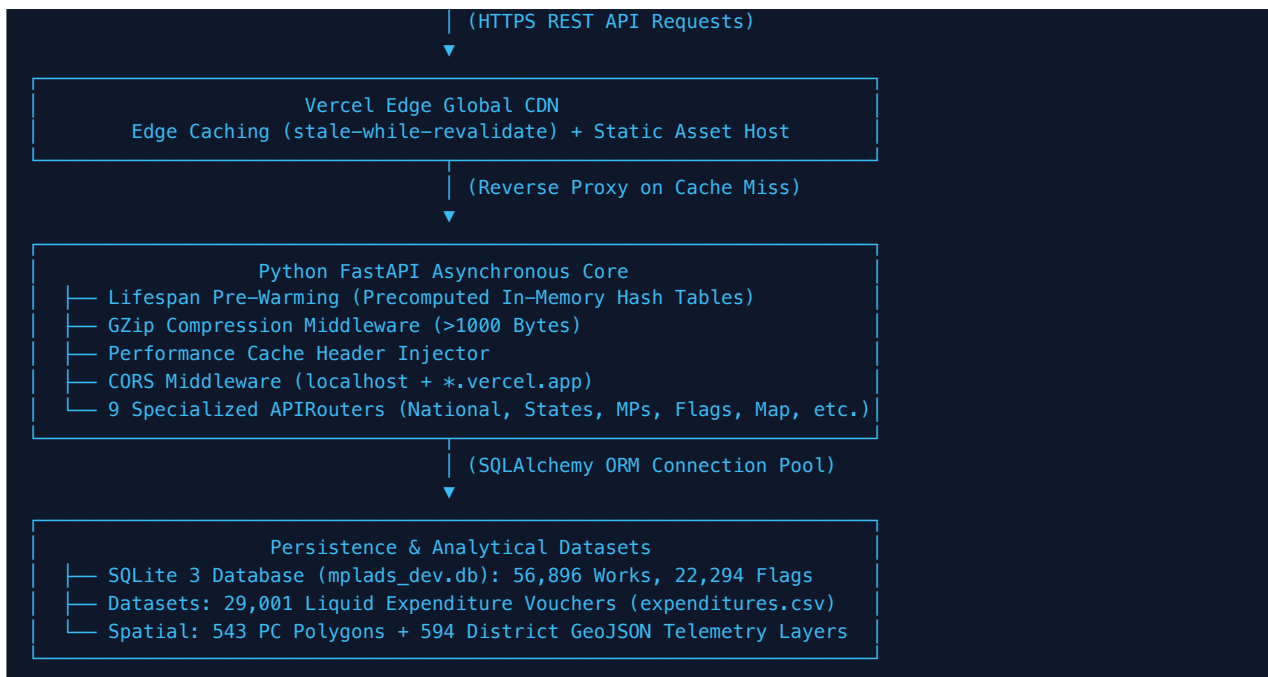
*Triggers Red Flag when  $\Delta C > 25\%$ .*

## Section 7: Tech Stack & Specific Architectural Rationale

- **Python 3.11/3.14 (FastAPI):** Chosen for native asynchronous I/O and seamless integration with numerical data science libraries ( `pandas` , `scikit-learn` , `numpy` ).
- **FastAPI (ASGI):** Chosen over Django/Flask because auto-generated OpenAPI schemas match frontend TypeScript interfaces, and its asynchronous request loop achieves sub-50ms latencies under heavy query loads.
- **SQLite 3 ( `mplads_dev.db` ) / SQLAlchemy ORM:** Chosen for self-contained, zero-configuration local persistence that deploys without external database server overhead during evaluations.
- **React 18 + Vite 5:** Chosen for client-side routing speed, Hot Module Replacement (HMR), and rapid rendering of complex stateful tables.
- **Tailwind CSS:** Chosen for utility-first styling to build an institutional, government-grade design system without heavy component library runtime overhead.
- **Leaflet & React-Leaflet:** Chosen for lightweight client-side rendering of multi-polygon GeoJSON boundaries (543 PCs) with smooth choropleth transitions.
- **Zustand:** Chosen over Redux for boilerplate-free global state management of user roles and cached filters.
- **Vercel Edge Network:** Chosen for global CDN caching of API payloads using HTTP `stale-while-revalidate` headers.

## Section 8: System Architecture & Data Flow

Citizen / Auditor Browser Layer  
React 18 + Vite + TypeScript + Tailwind + Leaflet GIS



### End-to-End Request Trace (Single User Request):

- 1. User Interaction:** An auditor on `/audit` filters for *Cost Overruns* in *Andhra Pradesh*.
- 2. Client Dispatch:** React dispatches: `GET /api/flags?detector=cost_overrun&state=Andhra+Pradesh&page=1`.
- 3. Edge Inspection:** Vercel Edge CDN evaluates the request; on cache miss, it forwards to the Python ASGI server.
- 4. Middleware Execution:** Starlette middleware evaluates CORS, sets caching headers, and hands the request to `webapi/routers/flags.py`.
- 5. Database Execution:** The router executes an indexed SQL query:

```
SELECT * FROM anomalies
WHERE detector_type = 'cost_overrun' AND state = 'Andhra Pradesh'
ORDER BY severity DESC LIMIT 50 OFFSET 0;
```

- 6. Serialization & Compression:** SQLAlchemy models are serialized into Pydantic models; `GZipMiddleware` compresses the payload by ~75%.
- 7. Client Render:** The browser decompresses the JSON in ~40ms, updates the Zustand store, dismisses skeletons, and paints the table.

## Section 9: The 15 Detectors (Forensic Fraud Checks)

### D1: Multivariate Statistical Outlier Screening ( `detector_01_unusual_patterns.py` )

- Input Data:** `works` table ( `sanctioned_amount` , `duration_days` , `category` , `district` ).
- Rule/Logic:** Applies an **Isolation Forest** multivariate algorithm to detect statistical outliers in financial and temporal profiles that deviate abnormally from category medians.
- Output:** Anomaly record with continuous anomaly score (e.g., `-0.81` ), feature importance breakdown, and Red/Amber tier flag.

### D2: Cross-Year Duplicate Scope & Text Similarity ( `detector_02_duplicate_works.py` )

- Input Data:** `works` table ( `work_description` , `district` , `sanctioned_amount` , `financial_year` ).

- **Rule/Logic:** Computes character and token **TF-IDF n-gram vectorization** with Cosine Similarity ( $>0.85$ ) across works within the same district to detect double billing under MPLADS Guidelines Clause 3.4.
- **Output:** Duplicate scope flag with matched target `work_id` and text similarity confidence score.

### D3: CPWD Cost Overrun Benchmark Analysis ( `detector_03_cost_overruns.py` )

- **Input Data:** `works.work_description` , `works.sanctioned_amount` , and `cpwd_benchmark_rates.csv` .
- **Rule/Logic:** Regular expressions extract physical quantities and units (e.g., "120 mtrs CC Road"). Computes actual unit rate and compares against CPWD Delhi Schedule of Rates (DSR 2023) ceiling (base rate + inflation + 25% tolerance).
- **Output:** Flag containing extracted unit, billed unit rate, allowed ceiling, excess billed rupees, and statutory schedule citation.

### D4: Payment Record & Disbursement Verification ( `detector_04_ghost_works.py` )

- **Input Data:** `works` table joined with `payment_vouchers` / `expenditures.csv` .
- **Rule/Logic:** Identifies works certified as "Completed" on the portal that have zero corresponding liquid treasury disbursement records or missing completion dates.
- **Output:** Advisory flag indicating unverified physical execution or ghost voucher risk under MPLADS Guidelines Clause 8.3.

### D5: Tender Splitting & Threshold Smurfing ( `detector_05_bill_splitting.py` )

- **Input Data:** `works` table ( `sanctioned_amount` , `sanction_date` , `work_description` , `implementing_agency` ).
- **Rule/Logic:** Flags identical or contiguous works sanctioned within a 7-day window whose individual values sit immediately below statutory e-tendering approval thresholds (e.g., ₹9.9 Lakhs vs ₹10 Lakh ceiling) to evade GFR 2017 Rule 157.
- **Output:** Tender splitting alert grouping clustered work IDs with cumulative evaded value.

### D6: Statutory Execution Duration & Stalled Works ( `detector_06_delay_violation.py` )

- **Input Data:** `works.recommendation_date` , `works.sanction_date` , `works.completion_date` .
- **Rule/Logic:** Evaluates time deltas against the mandatory 365-day statutory execution norm. Identifies stalled funds where projects remain incomplete  $>730$  days without formal extension.
- **Output:** Delay violation record displaying total days overdue and stalled public capital.

### D7: Fiscal Year-End March Rush & Dumping ( `detector_07_timing_anomaly.py` )

- **Input Data:** `works.completion_date` , `works.sanction_date` , `expenditures.expenditure_date` .
- **Rule/Logic:** Calculates the proportion of an MP's or district's annual expenditure authorized between March 25 and March 31. If March accounts for  $>60\%$  of annual turnover (vs. expected  $\sim 8.3\%$  monthly baseline), flags fiscal dumping.
- **Output:** Timing anomaly flag citing Public Accounts Committee (PAC) budget exhaustion risk.

### D8: Same-Day Batch Completion Screening ( `detector_08_bulk_completion.py` )

- **Input Data:** `works.completion_date` , `works.implementing_agency` , `works.district` .
- **Rule/Logic:** Flags instances where a single Implementing Development Agency (IDA) certifies  $>5$  distinct major civil infrastructure projects as completed on the exact same calendar date.
- **Output:** Paper sign-off alert highlighting bulk administrative rubber-stamping without physical site verification.

### D9: Benford's Law & Round-Number Forensic Screen ( `detector_09_benford_anomaly.py` )

- **Input Data:** `works.sanctioned_amount` across an MP's or agency's complete portfolio.
- **Rule/Logic:** Evaluates the distribution of first and second leading digits against the logarithmic Benford's Law curve ( $\log_{10}(1 + 1/d)$ ) and computes the proportion of suspicious round-number allocations (e.g., exactly ₹5,00,000 or ₹10,00,000).



- **Output:** Forensic accounting flag highlighting non-estimate arbitrary budget allocation.

#### D10: Vague Description & Scope Ambiguity Screen ( `detector_10_vague_description.py` )

- **Input Data:** `works.work_description` .
- **Rule/Logic:** Evaluates character length (<15\$ characters) and tests for missing fundamental scope entities: asset type, physical location, and quantifiable dimension (e.g., entries reading simply "Development work" or "Misc repair").
- **Output:** Scope ambiguity flag citing CAG performance audit non-verifiability standards.

#### D11: Category-Cost Engineering Plausibility Bounds ( `detector_11_plausibility_mismatch.py` )

- **Input Data:** `works.category` , `works.sanctioned_amount` .
- **Rule/Logic:** Evaluates project costs against absolute physical engineering min/max bounds per category (e.g., Drinking Water Borewell: ₹25,000 to ₹5,00,000; Community Hall: ₹3,00,000 to ₹1,50,00,000).
- **Output:** Plausibility mismatch flag identifying severe over- or under-budgeting.

#### D12: Documentary Verification Gap Forensics ( `detector_12_verification_gap.py` )

- **Input Data:** `measurement_books` , `geotagged_photos` metadata tables.
- **Rule/Logic:** Identifies projects marked completed that lack linked digitized Measurement Book (MB) entries or mandatory geo-tagged commencement/completion photographs.
- **Output:** Advisory audit flag indicating non-compliance with e-SAKSHI digital verification rules.

#### D13: Implementing District Authority (IDA) Portfolio Profiling ( `detector_13_ida_risk.py` )

- **Input Data:** Aggregation of all works grouped by `implementing_agency` .
- **Rule/Logic:** Calculates composite risk scores based on historical anomaly concentration, delay frequency, and unspent balances.
- **Output:** Institutional risk index ranking IDAs into High, Medium, and Low risk bands.

#### D14: Member of Parliament Portfolio Concentration ( `detector_14_mp_risk.py` )

- **Input Data:** Grouping of works by `mp_id` and `vendor_id / implementing_agency` .
- **Rule/Logic:** Measures single-agency or single-vendor allocation concentration (Herfindahl-Hirschman Index \$HHI > 0.40\$) and expenditure velocity. Strict exclusion of all demographic/political attributes.
- **Output:** Representative portfolio concentration metric highlighting geographic or vendor favoritism.

#### D15: Identical Repetitive Cost Matching / Copy-Paste Estimates ( `detector_15_copy_paste_pricing.py` )

- **Input Data:** `works` table ( `sanctioned_amount` , `work_description` , `district` ).
- **Rule/Logic:** Detects large clusters of disparate projects sanctioned at the identical non-standard rupee value (e.g., 20 projects in different locations all sanctioned at exactly ₹11,58,000).
- **Output:** Formulaic budgeting flag indicating absence of actual site-specific engineering estimates.